

Hands-on Lab

Atlas Auto Embedding & Vector Search 實戰

目標: 對 MongoDB 有基礎概念、想透過 MongoDB Atlas 一鍵體驗語意搜尋 (Semantic Search) 的開發者 / 架構師

資料: 交通部觀光署「中文版餐廳資料」

預估時間: 30-60 分鐘。

0. Lab 概覽

本 Lab 將帶領你完成：

1. 設定 **Auto Embedding**: 透過 Atlas UI 或 Compass 來建立 Auto Embedding 的 Vector Search Index, 由 Atlas 自動呼叫 Voyage AI 產生並同步向量。
2. 設定 **Text Search Index**: 透過 Atlas UI 或 Compass 來建立 Text Search Index。
3. 體驗 **Text Search, Vector Search, Hybrid Search**: 透過本次 workshop 網頁體驗不同搜尋方式所查出資料的結果。
4. 體驗 **Reranker** 優化查詢: 透過本次 workshop 網頁體驗搭配 Reranker 對於使用者語意的解讀所查出資料的結果。

重點概念: Auto Embedding 由 Atlas 在「索引建立」與「查詢」時, 自動透過 Voyage AI 產生向量, 開發者不再需要自行維護 embedding pipeline、API Key、retry/back-off 等基礎設施。

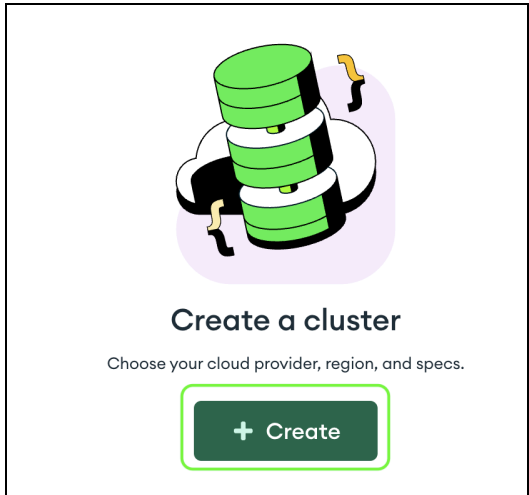
1. 前置需求 (Prerequisites)

項目	要求
MongoDB Atlas 叢集	M0 以上 Auto Embedding Public Preview 期間支援 M0
MongoDB 版本	8.0 以上 (含 Search & Vector Search 功能)
MongoDB Compass	1.46 以上 (內建 Auto Embedding 查詢樣板, COMPASS-10334 已合併)
網路	你的開發環境可連到 Atlas 叢集

2. Step 1 — MongoDB Atlas 環境建立

2.1 建立 MongoDB M0 Cluster

1. 在已經建立好的 Project 中，點選 [+Create] 來建立一個測試用的 Cluster 環境



2. 建立環境中，調整以下設定：

- 選擇 [Free]，表示採用 M0 規格
- 填入 [Name]，設定 Cluster Name，此欄位日後不可更改
- 選擇 [AWS] 的 [Singapore (ap-southeast-1)]
- 確認 [Preload sample dataset] 不勾選
- 按下 [Create Deployment] 完成 Cluster 建立

Deploy your cluster

Use a template below or set up advanced configuration options. You can also edit these configuration options once the cluster is created.

☐ **M10** **\$0.09/hour**

Dedicated cluster for development environments and low-traffic applications.

STORAGE	RAM	vCPU
10 GB	2 GB	2 vCPUs

☐ **Flex** **From \$0.011/hour**
Up to \$30/month

For development and testing, with on-demand burst capacity for unpredictable traffic.

STORAGE	RAM	vCPU
5 GB	Shared	Shared

☒ **Free**

For learning and exploring MongoDB in a cloud environment.

STORAGE	RAM	vCPU
512 MB	Shared	Shared

✔ **Free forever!** Your free cluster is ideal for experimenting in a limited sandbox. You can upgrade to a production cluster anytime.

Configurations

Name
You cannot change the name once the cluster is created.

workshop

Provider

☒ AWS ☐ Google Cloud ☐ Azure

Region

☒ Singapore (ap-southeast-1) ★

☐ Recommended ☒ Low carbon emissions

Quick setup

☒ Automate security setup ⓘ

☐ Preload sample dataset ⓘ

3. 設定 Security:

- 建立一組 MongoDB 資料庫連線使用的帳號與密碼, 建議以英數字為主, 不要採用符號
- 按下 [Create Database User] 完成資料庫使用者帳號密碼建立
- 按下 [Close] 關閉此頁面

Connect to workshop

1

2

3

Set up connection securityChoose a connection methodConnect

You need to secure your MongoDB Atlas cluster before you can use it. Set which users and IP addresses can access your cluster now. [Read more](#)

1. Add a connection IP address

✓ Your current IP address (61.222.54.230) has been added to enable local connectivity. Only an IP address you add to your Access List will be able to connect to your project's clusters. Add more later in [Network Access](#).

2. Create a database user

This first user will have [atlasAdmin](#) permissions for this project.

We autogenerated a username and password. You can use this or create your own.

i You'll need your database user's credentials in the next step. Copy the database user password.

Username
test

Password
.....
SHOW

Copy

Create Database User

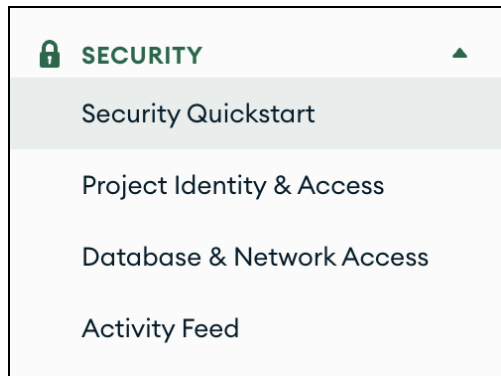
Close

Choose a connection method

2.2 設定網路環境

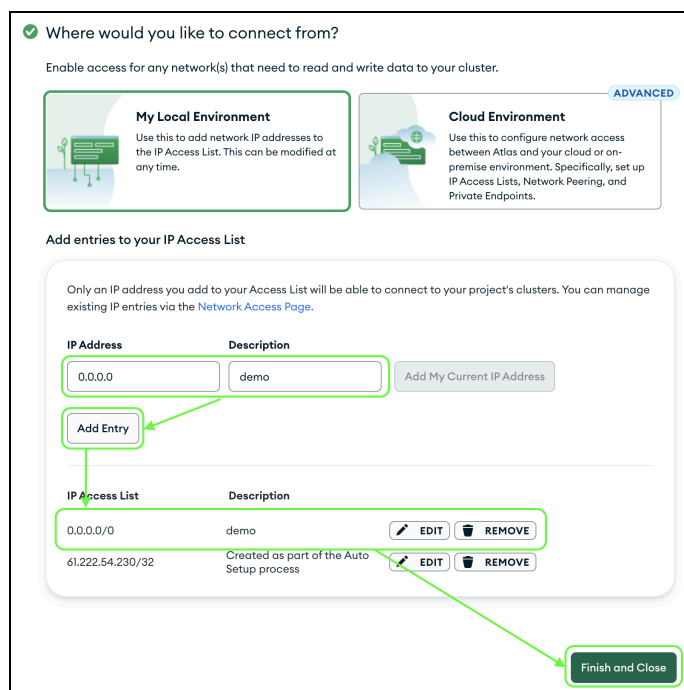
由於本次 workshop 需透過遠端將測試資料載入與進行後續資料測試，學員的 Cluster 需開放網路連線確保連線正常。

1. 點選 Atlas UI 左手邊的 [Security Quickstart]



2. 增加以下設定：

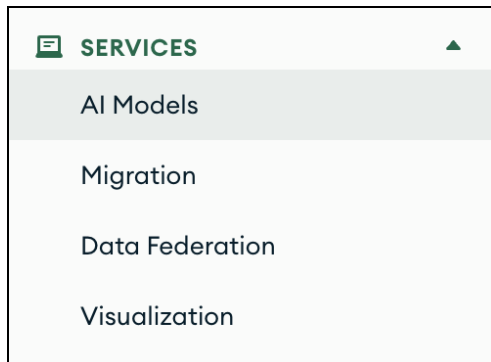
- 增加一組 0.0.0.0 的防火牆設定
- 按下 [Add Entry]
- 確認新的設定已經增加後，按下 [Finish and Close]



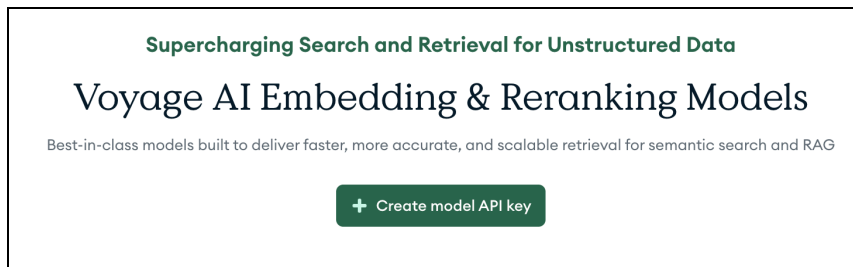
2.3 產生 VoyageAI API Key

由於本次 workshop 會呼叫 Reranker，需先建立一組 VoyageAI API Key 供後續使用。

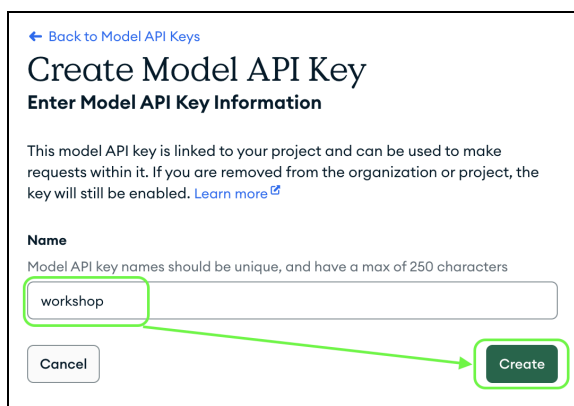
1. 點選 Atlas UI 左手邊的 [AI Models]



2. 按下 [+ Create model API Key] 開啟建立畫面



3. 輸入 [Name] 後, 按下 [Create]



系統會自動產生一組 API Key, 按下 [Copy] 並自行儲存起來, 按下 [Done] 關閉頁面
此 API Key 在關閉頁面後就無法再顯示其內容, 若遺失該 Key, 只能重新再建立一組新的

[← Back to Model API Keys](#)

Create Model API Key

Copy Model API Key

Name
workshop

Project
demo

Model API Key

Copy

ⓘ Copy your model API key and store it in a secure location. After you leave this page, you won't be able to view it again. If you lose your API key, you'll need to generate a new one.

Done

3. Step 2 — 載入測試資料

3.1 進入 Workshop 介面

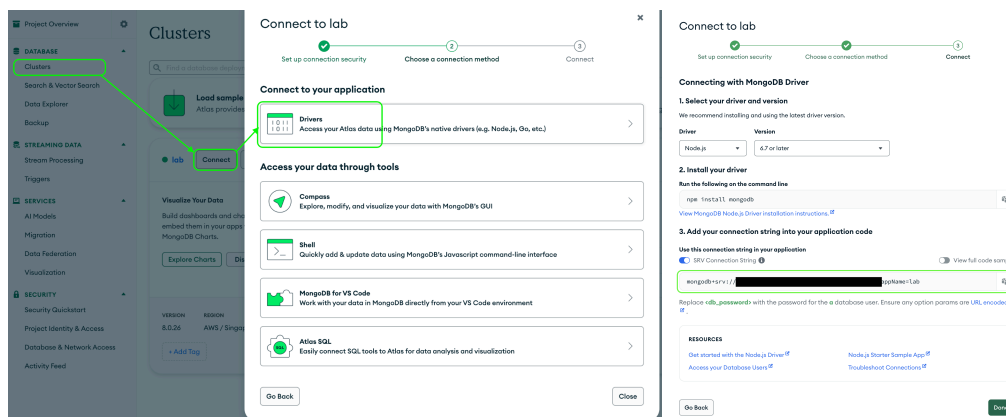
1. 透過 Browser 連接: <https://mdb.link/auto-emb>



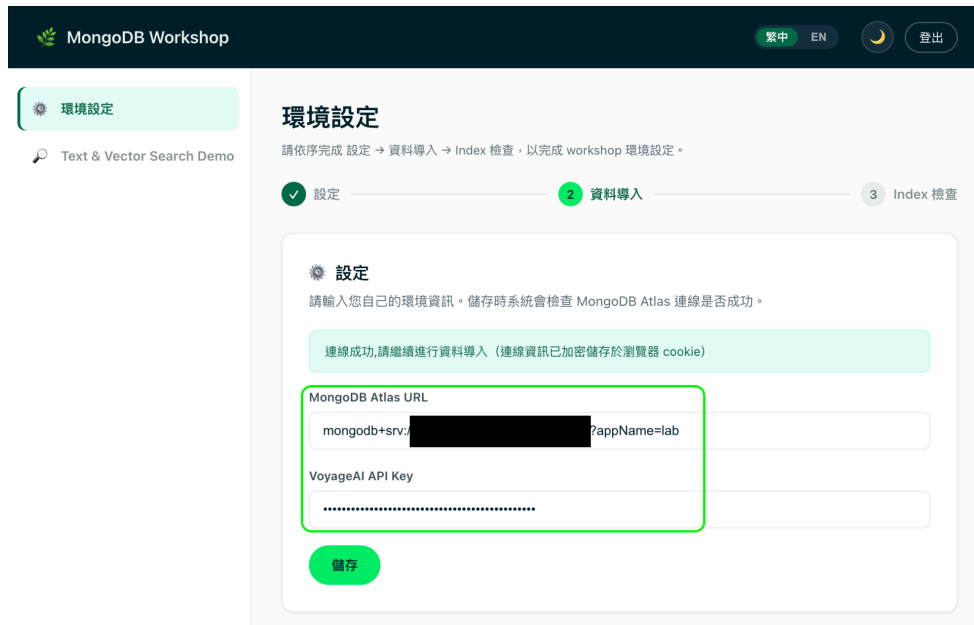
2. 輸入今日的 Secure Code 並按下登入

3.2 設定環境資訊

1. 取得步驟 2.1 的 Cluster 連線資訊, 點擊 Cluster 中的 [Connect], 選擇 [Drivers] 中取得



2. 將連線字串與步驟 2.3 產生 VoyageAI API Key, 同時填入 Workshop 介面中, 按下[儲存] 由系統驗證連線, 確認連線正確後, Workshop 介面會開啟 [資料導入] 區塊



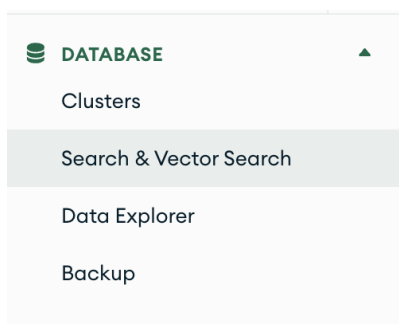
3. 點擊 [資料導入], 由系統將測試資料寫入 Atlas



4. Step 3 — 建立 Index

4.1 建立 Auto Embedding Vector Search Index

1. 點選 Atlas UI 左側 [Search & Vector Search]



2. 點選 [+ Create Search Index]

Search & Vector Search

Search & Vector Search

For AI- and relevance-based search applications

Performant and scalable full-text and semantic search as a native database capability.

[+ Create Search Index](#)

3. 選擇 [Vector Search] 與 [Automated Embedding]

[← Back to Search Indexes](#)

Create a Vector Search Index



Start Your Index Configuration

Search Type
Which Search type should I use?

Atlas Search
Full-text search for relevance-based app features.

Vector Search
For semantic search and AI applications.

How do you want to set up your vector data?
[Learn more about different data setup options](#)

Automated Embedding NEW START FREE
MongoDB generates and manages embeddings for your text fields – no external models or APIs needed.

Bring your own embeddings
Use vectors you've already generated or plan to generate with your own models or APIs.

4. 設定資料來源等

- Index 名稱: restaurant_auto_index
- Database & Collection: workshop.restaurant
- Configuration Method: Visual Editor

完成後點選 [Next]

Index Name and Data Source

Search indexes are specific to a database and collection.

At this time, search indexes cannot be created for time series collections.

Index Name

autoembed_index

Database and Collection

Search for database or collection ...

workshop

restaurant

Configuration Method

Select how you would like to configure your index. You can also define indexes through the [Atlas Admin API](#), [Atlas CLI](#), [MongoDB Shell](#), or supported [Drivers](#).

Visual Editor
Define index definitions in a more guided experience.

JSON Editor
Edit the raw index definition with an embedded JSON editor.

☒ I understand that token usage will be billed to my account even if no payment method is on file. If the balance is not paid, my organization may be suspended. [Add payment method](#)

Cancel

Next

5. 設定 Automated Embedding 欄位

- Path: embedding_text
- Embedding Model: voyage-4

完成後點選 [Next]

AutoEmbed Field

Select a text field for semantic search and MongoDB will auto-generate vector embeddings.

Modality text

Path

embedding_text

Embedding Model

voyage-4

recommended

Advanced

ADD ANOTHER AUTOEMBED FIELD

Filter Field

Optionally index values to pre-filter your data. [When should I use filter fields?](#)

Path

Type full path or select from list

ADD ANOTHER FILTER FIELD

Back

Cancel

Next

6. 最後按下 [Create Vector Search Index]

7. 等待 Vector Search Index 建立完成

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size	Required Memory
workshop	restaurant	restaurant_auto_index	PENDING View status details	✖	vectorSearch	"embedding_text"	0 (0%) indexed of 4,292	0B	0B

4.2 建立 Text Search Index

1. 點選 [Create Search Index], 選擇 [Atlas Search], 並輸入以下設定

- Index Name: restaurant_sindex
- Database & Collection: workshop.restaurant
- Configuration Method: Visual Editor

完成後點選 [Next]

Search Type

Which Search type should I use?

Atlas Search

Full-text search for relevance-based app features.

Vector Search

For semantic search and AI applications.

Index Name and Data Source

Search indexes are specific to a database and collection.

At this time, search indexes cannot be created for time series collections.

Index Name

restaurant_sindex

Database and Collection

Search for database or collection ...

workshop

restaurant

Configuration Method

Select how you would like to configure your index. You can also define indexes through the [Atlas Admin API](#), [Atlas CLI](#), [MongoDB Shell](#), or supported [Drivers](#).

Visual Editor

Define index definitions in a more guided experience.

JSON Editor

Edit the raw index definition with an embedded JSON editor.

2. Review Search Index 中, 點選 [REFINE YOUR INDEX] 修改 Index 相關設定

Create a Search Index

Setup

2Review

Review Search index "restaurant_index" for [workshop.restaurant](#)

VIEW JSONREFINE YOUR INDEX

You can edit these settings at any time to fine tune relevance and improve search performance. [Learn how to refine your index](#)

Index Configurations

Index Analyzer	Specifies how text is processed and tokenized when building the search index.	lucene.standard
Search Analyzer	Defines how search queries are processed and tokenized.	lucene.standard
Dynamic Mapping	Automatically maps supported data to field mapping types, including new fields and fields with polymorphic data.	On
Field Mappings	Configures specific fields with data types, analyzers, and input parameters. Field-level definitions override index-level definitions.	None
Stored Source Fields	Selects which fields from the collection are stored in the index for retrieval alongside search results. See Use Cases and Performance Considerations for details.	None
Synonyms Mappings	Respects defined word equivalents by accounting for similar terms (like "car" and "automobile") to improve search results.	None

Back

Cancel

Create Search Index

3. 在修改 Search Index 設定中, 將 Index Analyzer 修改為 lucene.chinese, 並按下 [Save Changes]

Index Configurations

Dynamic Mapping

Automatically maps supported data to field mapping types, including new fields and fields with polymorphic data. Use dynamic mappings if your schema changes regularly or is unknown, or when experimenting with Atlas Search. [Learn more](#)

Index Analyzer

Specifies how text is processed and tokenized when building the search index.

lucene.chinese

lucene.bengali

lucene.brazilian

lucene.bulgarian

lucene.catalan

✓ lucene.chinese

lucene.cjk

lucene.czech

lucene.danish

lucene.dutch

Search Analyzer

Defines how search queries are processed and tokenized.

lucene.chinese

on.

Query on: address

123 St.

We only support analysis for lucene.standard, lucene.keyword, lucene.whitespace and lucene.simple analyzers. Please let us know what you'd like to see [here](#)

4. 最後按下 [Create Search Index]

4.4 驗證 Index 建立成功

開啟 Workshop 介面，點擊 [ Index 檢查] 區塊中的 [檢查 Index] 按鈕，確認狀態從 [未建立] -> [建置中] -> [可用]，頁面上跳出 [環境設定已完成]

 **Index 檢查**

檢查 workshop.restaurant collection 是否建立 Vector Index (restaurant_auto_index) 與 Search Index (restaurant_sindex)，且兩者皆需為「可用 (可查詢)」狀態才算通過。

兩個 Index 皆已建立且為可用(可查詢)狀態

restaurant_auto_index
Vector Index · 狀態：READY · queryable: 是

可用

restaurant_sindex
Search Index · 狀態：READY · queryable: 是

可用

檢查 Index

 **環境設定已完成**

設定、資料導入與 Index 皆已就緒，可以開始進行 Text & Vector Search。

下一步：前往 Text & Vector Search →

5. Step 4 — 測試 Text / Vector / Hybrid Search + Reranker

5.1 透過 Workshop 頁面進行測試

Text & Vector Search Demo

對 workshop.restaurant collection 進行關鍵字搜尋與向量搜尋，並可選擇啟用 Reranker。

查詢語句
我想吃廣東粥，但是不要在金門

關鍵字搜尋

向量搜尋 (voyage-4)

向量搜尋 (voyage-4-lite)

Hybrid Search

☒ 啟用 Reranker (rerank-2.5)

Limit 10 回傳筆數 (1~100)

numCandidates 100 僅用於 Vector / Hybrid Search (須 ≥ Limit)

向量搜尋 (voyage-4) + Reranker (rerank-2.5) - 共 10 筆

Vector Search 耗時: 405.8 ms Reranker 耗時: 325.6 ms

Aggregation Pipeline

Vector Search (\$vectorSearch)

使用 Atlas Vector Search 的 \$vectorSearch stage，以 Auto-Embedding 自動將查詢字串轉為向量後進行語意搜尋。

```
{
  "$vectorSearch": {
    "index": "restaurant_auto_index",
    "path": "embedding_text",
    "query": "我想吃廣東粥，但是不要在金門",
    "limit": 10,
    "numCandidates": 100,
    "model": "voyage-4"
  },
  "$addFields": {
    "_score": {
      "$meta": "vectorSearchScore"
    }
  }
}
```

複製後貼至 MongoDB Compass → Aggregations 標籤，或以 mongosh 執行：db.restaurant.aggregate(pipeline)

#1 李記廣東粥

店家名稱
李記廣東粥

店家描述
店裡粥品口味十分多元，粥滑順，料多鮮美，從海鮮類吻仔魚、蝦球、虱目魚丸粥、鮮肉類的燒鴨、豬肝、牛肉、雞絲粥，到蔬菜類的田園蔬菜粥，應有盡有。來碗熱呼呼、香噴噴的粥，真是人間一大享受。地址:970 花蓮縣花蓮市中山路682號電話:03-8566560GIS座標:121.59615, 23.99432

地址
花蓮縣花蓮市中山路682號

score: 0.5056 rerank: 0.7773



{ } 查看 JSON Raw Data

- 查詢區塊

- 將查詢語句輸入，可透過自然語言表達方式撰寫
- 本測試提供
 - 關鍵字搜尋 - 運用 restaurant_sindex 執行關鍵字搜尋
 - 向量搜尋 (voyage-4) - 運用 restaurant_auto_index 並搭配 voyage-4 執行向量搜尋
 - 向量搜尋 (voyage-4-lite) - 運用 restaurant_auto_index 並搭配 voyage-4-lite 執行向量搜尋，展示 Shared Embedding Space 優勢
 - Hybrid Search - 運用 Reciprocal Rank Fusion 同時執行關鍵字與向量搜尋
 - Reranker(reranker-2.5) - 當啟用時，上述四種搜尋後執行 reranker 操作
- Limit & numCandidates - 控制查詢回傳筆數(limit) 與向量搜尋時的比對筆數(numCandidates)

- Aggregation Pipeline 區塊

- 當執行查詢後，自動呈現系統執行的 Aggregation Pipeline

- 可將此 pipeline 複製後，貼至 Compass 模擬執行，步驟如下：
 - Compass 連線至 Cluster 後，選擇 workshop 資料庫中的 restaurant collection
 - 點選 [Aggregations] 開啟 Aggregation 編輯器
 - 點選右方的 [TEXT] 採用文字輸入模式
 - 將 Workshop 介面中複製的 Aggregation Pipeline 貼上
 - Compass 將自動執行產生右邊的結果

The screenshot displays the MongoDB Compass interface for the 'restaurant' collection in the 'workshop' database. The 'Aggregations' tab is active, showing a pipeline with three stages: \$vectorSearch, \$addFields, and \$project. The \$vectorSearch stage uses the 'restaurant_auto_index' with a query '我想吃廣東菜，但是不要在金門' and a limit of 10. The \$addFields stage adds a '_score' field with a '\$meta' value of 'vectorSearchScore'. The \$project stage projects the '_id', '_score', and 'name' fields. The 'PIPELINE OUTPUT PREVIEW' on the right shows sample documents with fields like '_id', 'name', 'description', 'add', 'location', and '_score'.

```
1 {
2   "$vectorSearch": {
3     "index": "restaurant_auto_index",
4     "path": "embedding_text",
5     "query": "我想吃廣東菜，但是不要在金門",
6     "limit": 10,
7     "numCandidates": 100,
8     "model": "voyage-4"
9   },
10 },
11 {
12   "$addFields": {
13     "_score": {
14       "$meta": "vectorSearchScore"
15     }
16   },
17 },
18 {
19   "$project": {
20     "name": 1,
21     "description": 1,
22     "add": 1,
23     "location": 1,
24     "_score": 1
25   }
26 }
27
28
```

PIPELINE OUTPUT PREVIEW

Sample of 10 documents

```
{
  "_id": ObjectId('6a2baf2a8256537b07da87c4'),
  "name": "創記廣東菜",
  "description": "位在著名的師長功母廟旁坊旁，開業已逾半世紀，為金門地區廣東老店之一。每天晚上熬煮新鮮的粥，經細火慢燉後成粥，口味相較於其他店家較為清淡。",
  "add": "金門縣金城鎮民光路一段45號",
  "location": Object
  "_score": 0.5057915449142456
}
```

```
{
  "_id": ObjectId('6a2baf2a8256537b07da87c2'),
  "name": "永春廣東菜",
  "description": "這家永春廣東菜，從祖父就開始經營，鼎盛時期，祖父與父親同時經營兩家店面，家的手藝與口味是代代相傳，而且與台灣的相識不一樣，看不到陳腔，是廣東。",
  "add": "金門縣金城鎮民光路162之1號",
  "location": Object
  "_score": 0.5057765245437622
}
```

```
{
  "_id": ObjectId('6a2baf2a8256537b07da87c5'),
  "name": "陳俊廣東菜",
  "description": "位於金門古陳埔兵署旁，五年前開業。老闆娘用心熬煮成一碗碗廣東粥，希望讓遠道而來的客人都能嚐到一頓傳統的粥。就因為如此，這家店的粥品最",
  "add": "金門縣金城鎮民光路162之1號",
  "location": Object
  "_score": 0.5057765245437622
}
```